Software Architecture and Design Specification

Project: Real-Time Chat Application (Socket Programming)

Version: 1.1

Authors: Team 5, PES University, Bangalore

Date: 07-10-2026

Status: Draft

Revision History

Version	Date	Author	Change Summary
1.0	04-10-2026	Team 5	Initial architecture and design specification derived from SRS v1.0
1.1	07-10-2026	Team 5	Added team member names, SRNs and roles

Approvals

Role	Name (SRN)	Signature / Date
Member 1: Authentication and Security (SRS 5.1, SEC-1 to SEC-8)	Gagan Eswar (PES1UG24CS165)	
Member 2: Messaging, Chat History and Performance (SRS 5.2, 5.5, PERF)	Chetha Samrutha (PES1UG24CS131)	
Member 3: Group Chat, Profile/Search and Usability (SRS 5.3, 5.7, UI)	Harsh Kalekar (PES1UG24CS183)	
Member 4: Presence, Status, Connection Handling and Reliability (SRS 5.4, 5.6)	Dhanush K S (PES1UG24CS148)	
Project Guide		

1. Introduction

1.1 Purpose

This document specifies the architecture and design of the Real-Time Chat Application (release 1.0). It translates the requirements of the Software Requirements Specification (SRS v1.0) into a structure of components, a security architecture, interaction flows, interface definitions and an error-handling approach that the team will implement and test.

1.2 Scope

Covers the complete system: the browser-based client, the Node.js application server (REST API and WebSocket server) and the MongoDB database. The functional areas are user registration and authentication, one-to-one messaging, group chat, presence and typing indicators, chat history, delivery status with unread notifications, and profile management with user search. Items listed as out of scope in the SRS (file sharing, voice/video, end-to-end encryption, native apps, message editing) are not designed here.

1.3 Audience

Developers (Team 5), testers deriving test cases from the design, the project guide and evaluators, and anyone who will maintain or extend the system.

1.4 Definitions

Term	Meaning
ADR	Architecture Decision Record, a short note recording one design decision and its rationale
JWT	JSON Web Token, a signed token proving that a user has authenticated
WS / WSS	WebSocket / WebSocket Secure (over TLS), RFC 6455
REST	Representational State Transfer, the style of the HTTP API
ODM	Object-Document Mapper (Mongoose) used to access MongoDB
STRIDE	Threat classification: Spoofing, Tampering, Repudiation, Information disclosure, Denial of service, Elevation of privilege
XSS	Cross-Site Scripting, injection of malicious scripts into a web page
TLS	Transport Layer Security, the encryption protocol behind HTTPS and WSS

2. Document Overview

2.1 How to use this document

Section 3 describes the architecture (component diagram, pattern, technology, risks, requirement traceability, security architecture). Section 4 describes the detailed design (sequence diagrams, API design, error handling, UX). Section 5 holds the glossary, references and tools. Requirement identifiers (REQ-n, PERF-n, SAF-n, SEC-n, BR-n) refer to SRS v1.0.

2.2 Related Documents

- Software Requirements Specification (SRS) v1.0, Team 5, 04-Oct-2026, including the Requirement Traceability Matrix (Appendix C).
- Software Test Plan (STP) v1.0, Team 5, which traces test cases back to the same requirement identifiers.

3. Architecture

3.1 Goals & Constraints

Goals:

- Low-latency delivery: at least 95% of messages displayed within 500 ms on a local network (PERF-1).
- Reliability: no loss of accepted messages (SAF-1); automatic recovery from short network drops (REQ-22).
- Security and privacy: authenticated, authorised and encrypted communication (SEC-1 to SEC-8).
- Usability: responsive single-page UI that works from 360 px mobile width upwards.
- Maintainability: modular code with separate auth, messaging, presence and data layers.

Constraints:

- Real-time communication must use WebSockets, not long-polling.
- Node.js back end with the ws library, Express and MongoDB; only free and open-source libraries.
- Single-server deployment, about 100 concurrent users (demo scale), no clustering in release 1.0.
- Passwords stored only as salted bcrypt hashes; secrets kept in environment variables.
- Mini-project schedule: the design must be simple enough to implement and test within the sprints.

3.2 Stakeholders & Concerns

Stakeholder	Main concerns	Addressed by
End users (registered users, group admins)	Instant delivery, privacy, easy interface, no lost messages	WebSocket gateway, persist-before-ack, TLS, responsive UI (3.4, 3.9, 4.5)

Stakeholder	Main concerns	Addressed by
Developers (Team 5)	Modularity, clear interfaces, testability	Layered structure with seven modules (3.4, 3.5)
Testers	Traceable requirements, observable behaviour	Traceability table (3.8), API contracts (4.3), error codes (4.4)
Project guide / evaluators	Correct use of socket programming, quality of documentation	Pattern rationale (3.5), diagrams (3.3, 4.2)
System administrator	Easy deployment, monitoring, backups	Environment-based configuration, health endpoint, logging (4.4)

3.3 Component (UML) Diagram

Figure 1 shows the components of the system and their dependencies. The seven business modules use the same names as the Architecture Reference column of the SRS traceability matrix (Appendix C of the SRS).

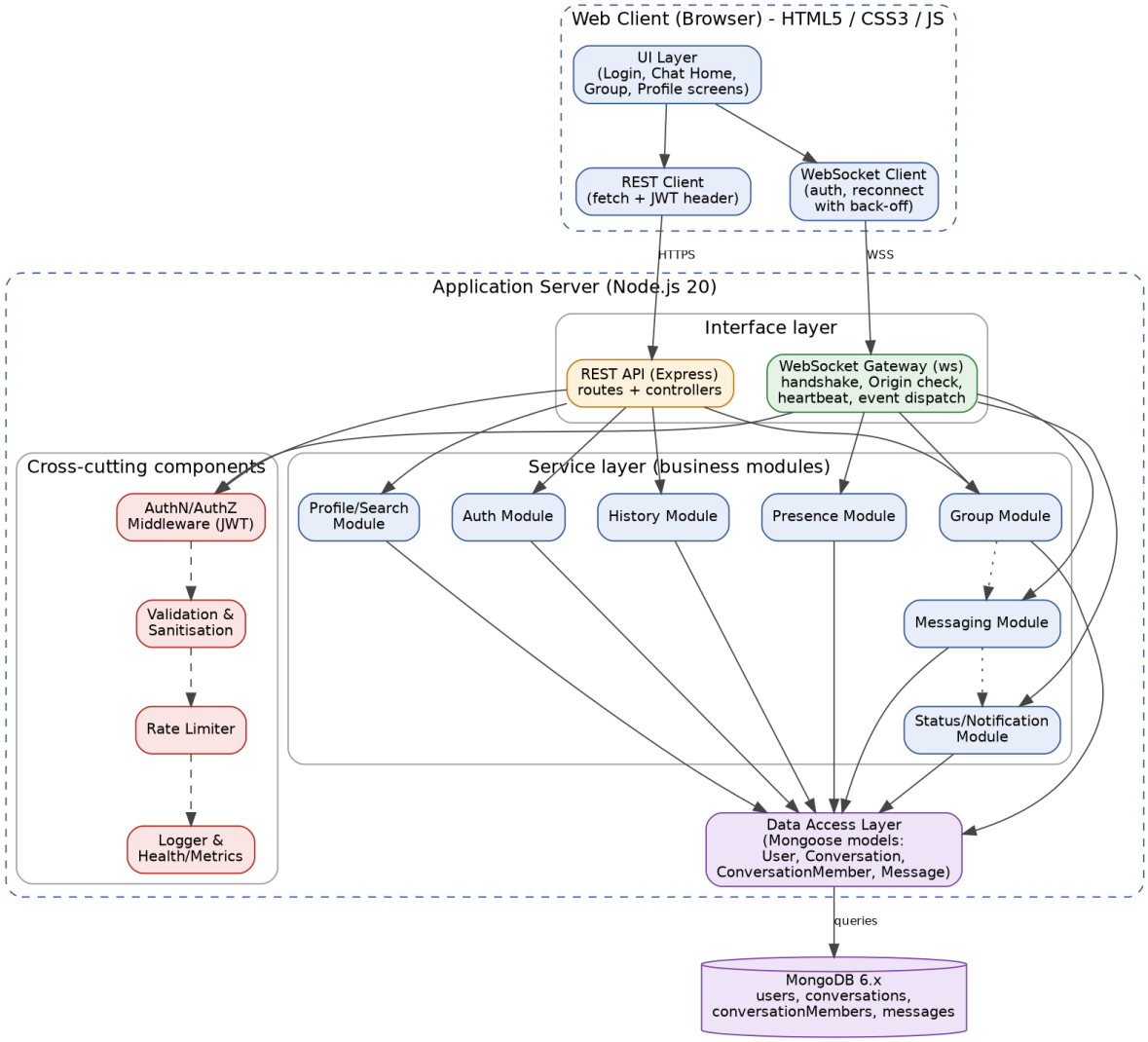


Figure 1: UML component diagram of the Chat Application

3.4 Component Descriptions

Component	Responsibility	Technology
UI Layer (client)	Renders Register/Login, Chat Home, Group, Profile and notification screens; sanitised rendering of messages; keyboard shortcuts	HTML5, CSS3, JavaScript (ES6)

Component	Responsibility	Technology
WebSocket Client	Opens the WSS connection, sends the auth event, sends/receives events, reconnects with exponential back-off (1 s up to 30 s), re-synchronises missed messages	Browser WebSocket API
REST Client	Calls the REST API with the JWT in the Authorization header	fetch API
REST API (Express)	Routes and controllers for registration, login, search, profile, groups and history; applies middleware	Node.js, Express 4.x
WebSocket Gateway	Accepts upgrade requests, checks Origin, waits for the auth event, keeps a map userId to open connections, sends ping every 30 s, dispatches events to modules	ws 8.x
AuthN/AuthZ Middleware	Verifies JWT, loads the caller identity, checks conversation membership before any read or write	jsonwebtoken 9.x
Validation & Sanitisation	Validates field formats and lengths; escapes HTML in message content on the server	validator, sanitising library
Rate Limiter	Limits each user to 20 messages per 10 seconds and throttles login attempts	In-memory token bucket
Logger & Health/Metrics	Structured logs without secrets; /health endpoint; counters for connections, failed logins and latency	pino or winston
Auth Module	Registration, password hashing, login, failed-attempt counting and account lock, token issue and verification, logout	bcrypt 5.x, jsonwebtoken
Messaging Module	Validates and stores one-to-one messages, delivers to online recipients, queues for offline recipients	ws, Mongoose
Group Module	Creates groups, adds/removes members, renames, leave, membership checks, group fan-out	Express, ws, Mongoose
Presence Module	Online/offline state, last-seen, typing events, heartbeat handling	ws
History Module	Paged retrieval of messages (50 per page) and of missed messages after reconnect	Express, Mongoose
Status/Notification Module	SENT, DELIVERED and READ transitions, unread counters	ws, Mongoose
Profile/Search Module	View/edit profile, change password, partial case-insensitive username search	Express, Mongoose
Data Access Layer	Mongoose models for User, Conversation, ConversationMember and Message; the only code that talks to the database	Mongoose 7.x
MongoDB	Persistent store with unique indexes on username and email and a compound index on (conversationId, sentAt)	MongoDB 6.x

3.5 Chosen Architecture Pattern and Rationale

Pattern: a client-server system whose server is a **layered monolith** (interface layer, service layer, data-access layer) with an **event-driven** WebSocket interface. Cross-cutting concerns (authentication, validation, rate limiting, logging) are implemented once as middleware and shared by the REST and WebSocket paths.

Rationale:

- Layering separates concerns, so each of the four team members can own modules (see SRS team profile) with few merge conflicts and test them in isolation (Maintainability, Testability).
- A monolith on one Node.js process fits the 100-user demo scale and the single-server constraint; it avoids network hops between services and keeps latency low (PERF-1).
- An event-driven message format ({"type", "payload"}) maps naturally to WebSocket full-duplex communication and keeps clients interoperable.
- Thin REST for request/response operations (register, login, history, profile) and WebSocket for push events gives each protocol the job it is best at.

Alternatives rejected:

- Microservices: too much deployment and coordination overhead for a four-person mini-project and a single server.
- HTTP long-polling: forbidden by the problem statement and higher latency.
- Peer-to-peer messaging: no central place for persistence, authorisation or history.

Architecture Decision Records

ID	Decision	Reason	Consequence
ADR-1	Layered monolith, single Node.js process	Scale and schedule of the mini-project	No horizontal scaling in v1.0; a Redis pub/sub broker can be added later (SRS 6.4 Scalability)
ADR-2	JWT is sent in the first WebSocket message (auth event) within 5 seconds of opening, not in the URL	Tokens in URLs leak into logs and browser history (SEC-2); the Origin header is still checked at upgrade (SEC-6)	Connections without a valid auth event are closed with code 4401 (REQ-6)
ADR-3	Message is written to MongoDB before the SENT ack is returned	No loss of accepted messages (SAF-1, REQ-23)	Slightly higher latency per message, within the 500 ms target
ADR-4	At-least-once delivery with de-duplication by message ID (clientMsgId + messageId)	Reconnection may re-send; duplicates must be ignored (SRS 6.5 domain requirements)	Client keeps a set of seen IDs per conversation
ADR-5	MongoDB with Mongoose	Flexible documents, simple paging with a compound index, free and open source	Referential integrity is enforced in the Data Access Layer
ADR-6	Server assigns timestamps and message IDs	Consistent ordering across clients (SRS 3.3)	Client clocks are never trusted

3.6 Technology Stack & Data Stores

Layer	Technology	Version
Client	HTML5, CSS3, vanilla JavaScript (ES6) single-page application	Latest 2 versions of Chrome, Firefox, Edge, Safari
Server runtime	Node.js	20 LTS or later
REST framework	Express	4.x
WebSocket	ws	8.x
Database	MongoDB with Mongoose ODM	6.x+ / 7.x+
Security libraries	bcrypt (cost 10 or higher), jsonwebtoken, dotenv, validator	5.x / 9.x
Transport security	TLS 1.2+ (self-signed for demo, Let's Encrypt when hosted)	
Quality tools	ESLint, Jest (unit/integration), Postman, Git	

Data stores (MongoDB collections):

Collection	Key fields	Indexes
users	username, email, passwordHash, displayName, statusMessage, createdAt, lastSeen, failedAttempts, lockedUntil	Unique: username (lower-case), email
conversations	type (PRIVATE/GROUP), name, createdBy, createdAt	createdBy

Collection	Key fields	Indexes
conversationMembers	conversationId, userId, role (ADMIN/MEMBER), joinedAt, lastReadMessageId	Unique compound (conversationId, userId); userId
messages	conversationId, senderId, content, sentAt, status	Compound (conversationId, sentAt desc)

Unread count of a conversation = number of messages in it with an ID greater than the member's lastReadMessageId and a sender other than the member (REQ-27). Field-level layouts are in SRS Appendix B.

3.7 Risks & Mitigations

Risk	Mitigation
Connection drops lose or duplicate messages	Persist before ack; reconnect with back-off; re-sync using ?after=lastMessageId; de-duplicate by message ID
Single server is a single point of failure	Graceful error messages (SAF-3); database backup before every demo (SAF-4); modular code that allows adding a broker later
Message flooding or abuse	Rate limit of 20 messages per 10 s per user (SAF-2); frame size cap of 4 KB
WebSocket concurrency bugs (race between disconnect and send)	One connection map owned by the gateway; integration tests with simulated clients; load test with 100 clients
Team members blocked by dependencies on each other's modules	API contracts in 4.3 agreed first; modules mocked in unit tests
TLS certificate or Origin misconfiguration in hosted demo	Configuration through .env.example; a deployment checklist; fallback to local demo over ws://
Learning curve of ws, JWT and Mongoose	Early spike in sprint 1; shared code style and reviews

3.8 Traceability to Requirements

Every requirement of the SRS is allocated to at least one component. The same allocation appears in the Architecture Reference column of the SRS traceability matrix.

Requirement(s)	Description (short)	Component(s)
REQ-1 to REQ-5	Register, validate, hash, login, lockout	Auth Module, REST API, Validation, Rate Limiter
REQ-6, REQ-7	Authenticated WebSocket, logout	WebSocket Gateway, Auth Module, AuthN Middleware
REQ-8, REQ-9	Open private chat, real-time delivery	Messaging Module, WebSocket Gateway
REQ-10, REQ-13	Length limits, XSS sanitising	Validation & Sanitisation, Messaging Module, UI Layer
REQ-11	Offline storage and delivery	Messaging Module, Data Access Layer
REQ-12	Message display with sender, time, status	UI Layer, Messaging Module
REQ-14 to REQ-18	Groups, fan-out, admin rights, membership check	Group Module, AuthZ Middleware, REST API, WebSocket Gateway
REQ-19, REQ-20	Presence and typing	Presence Module, WebSocket Gateway
REQ-21, REQ-22	Heartbeat, reconnection	WebSocket Gateway, Presence Module, WebSocket Client
REQ-23, REQ-24	Persistence, paged history	History Module, Messaging Module, Data Access Layer
REQ-25 to REQ-28	Status ticks, unread count, notifications	Status/Notification Module, UI Layer
REQ-29 to REQ-31	Profile, change password, user search	Profile/Search Module, REST API

Requirement(s)	Description (short)	Component(s)
PERF-1 to PERF-5	Latency, 100 connections, load times	WebSocket Gateway, Data Access Layer (indexes), UI Layer
SAF-1 to SAF-4	Durability, rate limit, graceful failure, backup	Messaging Module, Rate Limiter, Error handler, MongoDB
SEC-1 to SEC-8	Security requirements	Security architecture (3.9)

3.9 Security Architecture

Security objectives:

- **Confidentiality:** only the participants of a conversation can read its messages; passwords, tokens and personal data are never exposed.
- **Integrity:** messages cannot be forged, altered in transit or injected as active content (XSS, NoSQL injection).
- **Availability:** the service resists flooding and brute-force login.
- **Accountability:** security events (failed logins, lockouts, rejected tokens) are logged without sensitive values.

Trust boundaries: (1) browser to server over the public network, protected by TLS; (2) server process to MongoDB, restricted to a private/local connection with credentials from environment variables. All input crossing boundary (1) is untrusted.

Threat model (STRIDE):

STRIDE	Threat in this system	Control	Req.
Spoofing	Attacker impersonates a user or opens a WebSocket without credentials; cross-site WebSocket hijacking	bcrypt login; signed JWT (24 h); auth event required within 5 s; Origin check at handshake	SEC-1, SEC-6, REQ-6
Tampering	Message altered in transit; script injected into message content; NoSQL/query injection	TLS 1.2+ (wss/https); server and client sanitising; Mongoose schemas and parameterised queries; schema validation of every event	SEC-3, SEC-4, REQ-13
Repudiation	User denies sending a message or a login attempt	Server-assigned sender ID, timestamp and message ID; security event log with user ID and time	SEC-2, BR-5
Information disclosure	Reading another user's conversation; password hash or token leaked in logs or errors	Membership check on every read/write; hashes and tokens excluded from logs and API responses; generic "Invalid credentials"; no stack traces to clients	SEC-2, SEC-5, REQ-5
Denial of service	Message flooding, oversized frames, brute-force login, too many connections	Rate limit 20 msg / 10 s; 4 KB frame limit; 2,000-character message cap; account lock after 5 failures for 15 min; ping/pong clean-up of dead sockets	SAF-2, SEC-7
Elevation of privilege	Non-admin changes group membership; non-member posts to a group	Role check (ADMIN) on group management; membership check on messaging; user ID taken from the verified token, never from the request body	SEC-5, BR-4, REQ-18

Security design decisions:

- Passwords: bcrypt with cost factor 10 or higher and a per-password salt; minimum 8 characters with a letter and a digit.
- Tokens: JWT signed with a secret from the environment, 24-hour expiry; on logout the client discards it and the server closes its socket.

- Secrets and credentials: only in environment variables (.env, never committed); .env.example documents the names.
- Data protection: personal data (email, messages) is used only for the chat service and not shared with third parties (SEC-8); the design follows the OWASP Top 10.
- Security testing is planned in section 5.1 of the Software Test Plan.

4. Design

4.1 Design Overview

The server is organised into three layers. The interface layer (Express routes and the ws gateway) only parses requests and events, applies middleware and calls the service layer. The service layer holds the seven business modules. The data-access layer exposes Mongoose models and is the only code that reads or writes MongoDB. Each module has its own folder with routes/handlers, service and tests, which supports the team ownership split described in the SRS.

Real-time event contract. All WebSocket frames are UTF-8 JSON of the form {"type": <event>, "payload": {...}} with a maximum frame of 4 KB, as defined in SRS 3.2. Events: auth, message:send, message:new, message:status, typing, presence, group:update, error. Two small additions are made by this design: the server replies auth:ok after a successful auth, and message:status is also used by the server to acknowledge SENT to the sender.

Connection life cycle. Open (Origin check) then auth within 5 s then ONLINE then ping every 30 s, pong within 10 s then close or timeout then OFFLINE and last-seen update. The user is marked offline within 40 seconds of a broken connection (REQ-21).

Message life cycle. Client creates clientMsgId, sends message:send. Server validates, stores with status SENT and a server timestamp, acknowledges SENT, pushes message:new to online recipients, and receives DELIVERED and READ updates from recipients. Offline recipients receive the message after reconnect through history re-synchronisation.

Concurrency. Node.js runs the event loop on a single thread; shared state (connection map, typing timers, rate counters) is held in memory objects owned by the gateway and presence modules and is never shared across processes in v1.0.

4.2 UML Sequence Diagrams

Four flows that are specific to this project are shown: login with WebSocket authentication, sending a private message with delivery status, group creation and group messaging, and connection loss with reconnection.

Sequence 1: Login and WebSocket authentication (REQ-4 to REQ-6, REQ-19)

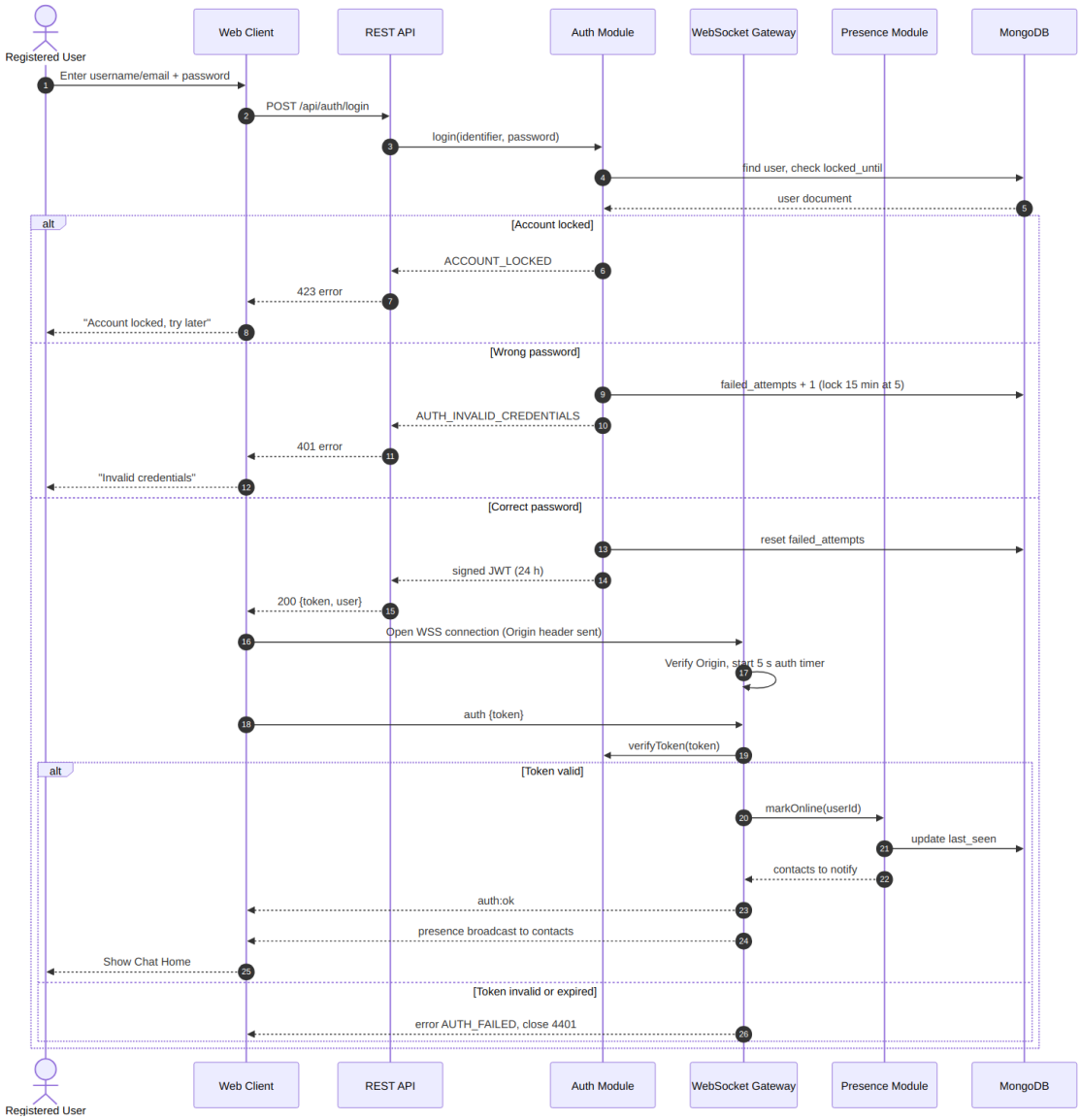


Figure 2: Sequence diagram, login and WebSocket authentication

Sequence 2: Send private message with delivery status (REQ-8 to REQ-13, REQ-23, REQ-25, REQ-26)

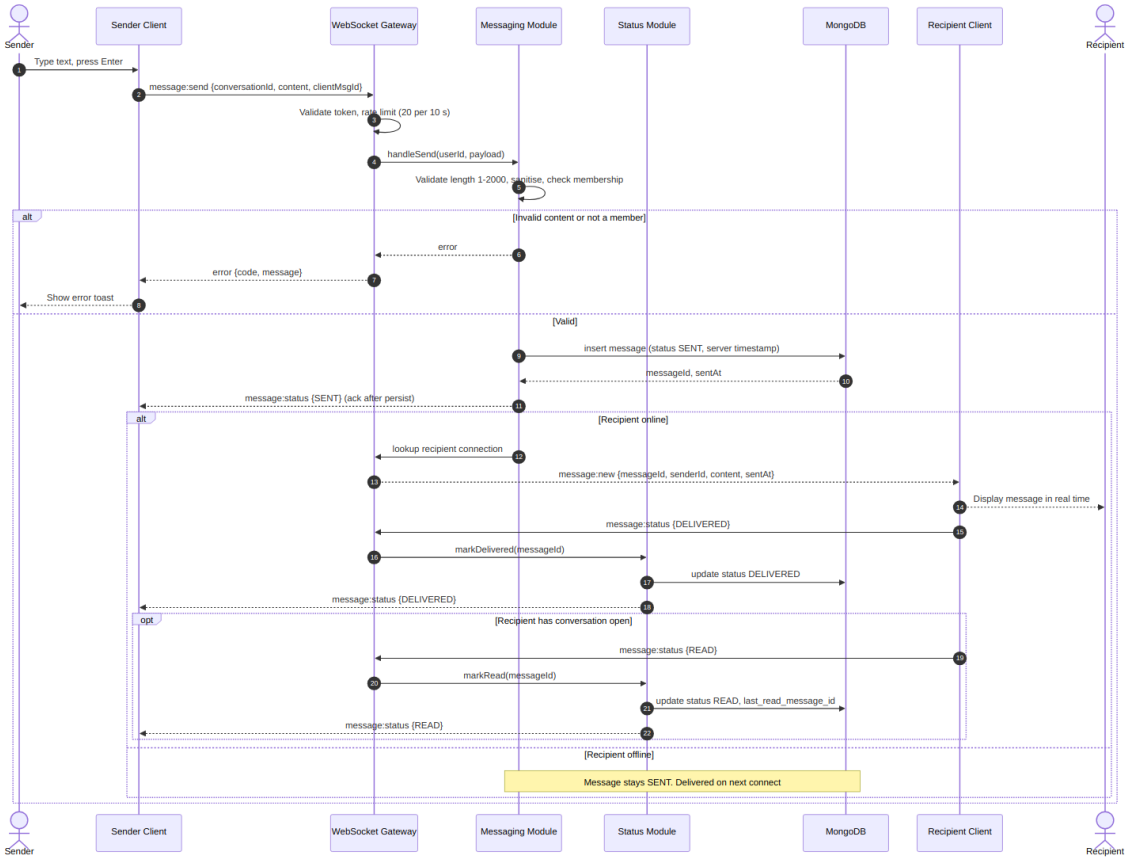


Figure 3: Sequence diagram, one-to-one message, delivered and read status

Sequence 3: Create group and send group message (REQ-14 to REQ-18)

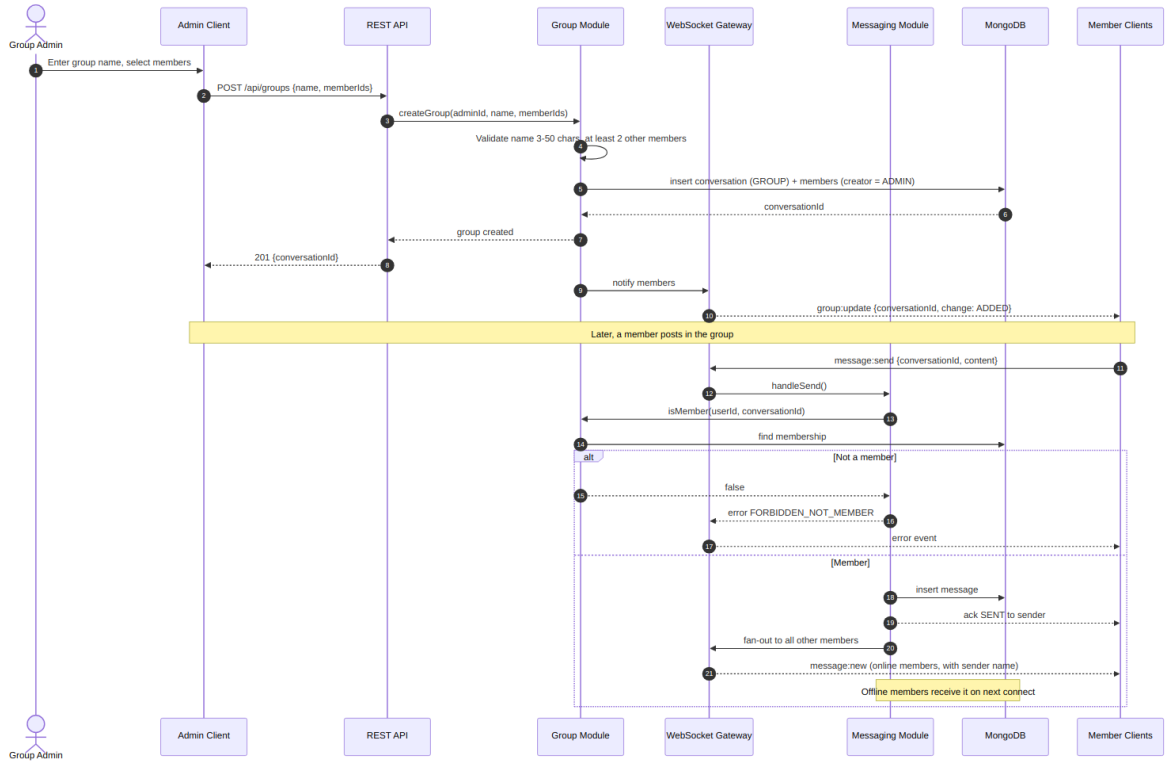


Figure 4: Sequence diagram, group creation and group message fan-out

Sequence 4: Heartbeat failure and reconnection (REQ-21, REQ-22, REQ-24)

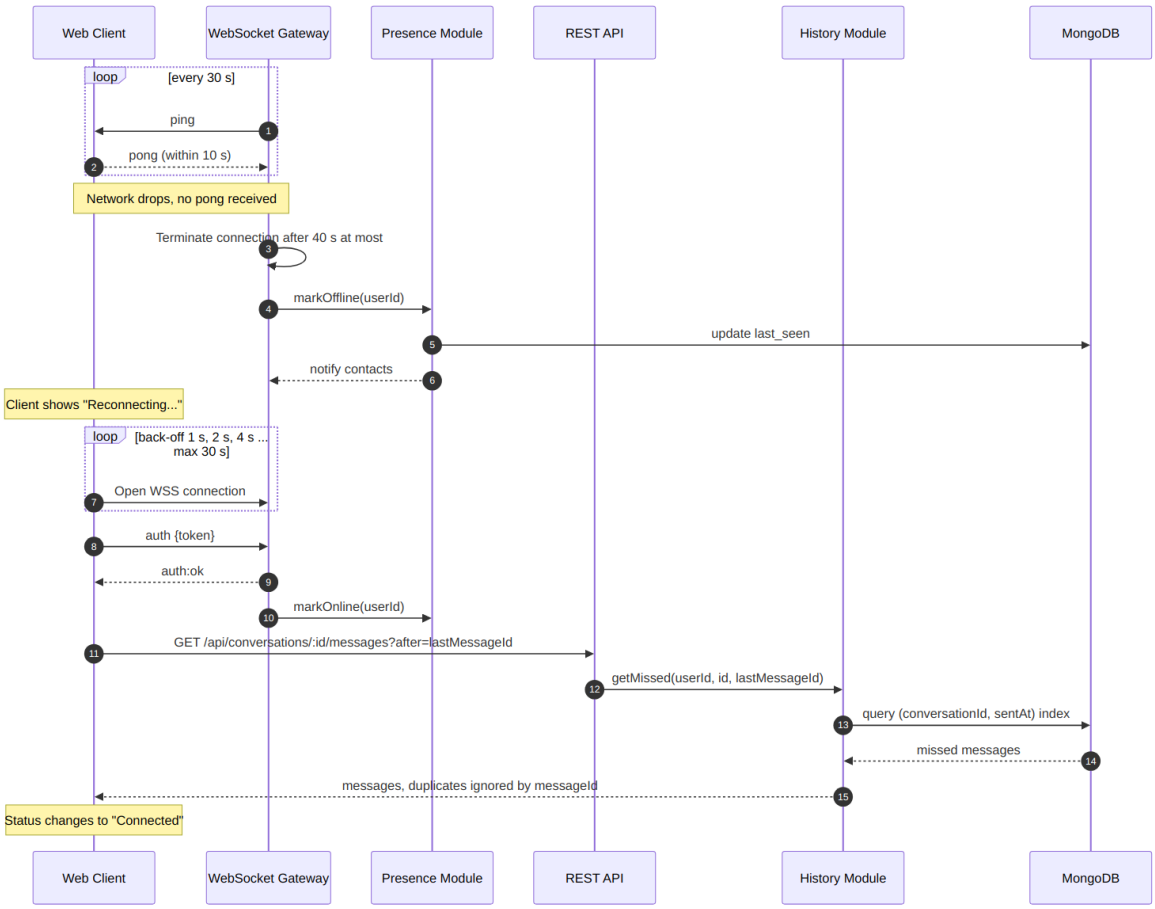


Figure 5: Sequence diagram, heartbeat, reconnection and re-synchronisation

4.3 API Design

The system exposes two interfaces: the REST API of the Auth, Profile/Search, Group and History modules, and the WebSocket event interface of the Messaging, Presence and Status modules. All REST bodies are JSON. Except for register and login, every REST endpoint requires the header Authorization: Bearer <JWT>. User identity always comes from the token.

Interface 1: REST API

Method and path	Purpose (Req.)	Request	Success response	Errors
POST /api/auth/register	Create account (REQ-1 to REQ-3)	{username, email, password}	201 {userId, username}	400 VALIDATION_ERROR, 409 DUPLICATE_USER
POST /api/auth/login	Login by username or email (REQ-4, REQ-5)	{identifier, password}	200 {token, expiresIn, user}	401 AUTH_INVALID_CREDENTIALS, 423 ACCOUNT_LOCKED
POST /api/auth/logout	Logout, close socket (REQ-7)	none	204	401 AUTH_TOKEN_INVALID
GET /api/users/search?q=	Partial, case-insensitive search (REQ-31)	query q (1-20 chars)	200 [{userId, username, displayName}]	400, 401

Method and path	Purpose (Req.)	Request	Success response	Errors
GET /api/users/me	View profile (REQ-29)	none	200 {username, email, displayName, statusMessage}	401
PATCH /api/users/me	Edit display name and status (REQ-29)	{displayName?, statusMessage?}	200 profile	400 VALIDATION_ERROR, 401
PUT /api/users/me/password	Change password (REQ-30)	{currentPassword, newPassword}	204	400 VALIDATION_ERROR, 401 AUTH_INVALID_CREDENTIALS
GET /api/conversations	List conversations with unread counts (REQ-27)	none	200 [{conversationId, type, name, lastMessage, unreadCount}]	401
POST /api/conversations	Open or create private chat (REQ-8)	{participantId}	200/201 {conversationId}	404 NOT_FOUND, 401
GET /api/conversations/:id/messages?before=&after=&limit=50	Latest 50, older pages, missed messages (REQ-24, REQ-22)	query: before or after message ID	200 {messages[], hasMore}	403 FORBIDDEN_NOT_MEMBER, 404
POST /api/groups	Create group (REQ-14)	{name, memberIds[]}	201 {conversationId}	400 VALIDATION_ERROR, 404
PATCH /api/groups/:id	Rename group (REQ-16)	{name}	200 group	403 FORBIDDEN_NOT_ADMIN, 400
POST /api/groups/:id/members	Add member (REQ-16)	{userId}	200 members	403 FORBIDDEN_NOT_ADMIN, 409, 404
DELETE /api/groups/:id/members/:userId	Remove member (REQ-16)	none	204	403 FORBIDDEN_NOT_ADMIN, 404
POST /api/groups/:id/leave	Leave group (REQ-16)	none	204	403, 404
GET /health	Liveness and DB check (SAF-3)	none	200 {status, uptime}	503

Example: POST /api/auth/login

Request: {"identifier": "alice", "password": "Secret123"}

Response 200: {"token": "<jwt>", "expiresIn": 86400, "user": {"userId": "...", "username": "alice", "displayName": "Alice"}}

Response 401: {"error": {"code": "AUTH_INVALID_CREDENTIALS", "message": "Invalid credentials"}}

Interface 2: WebSocket events

Event (direction)	Payload	Validation and behaviour	Error codes
auth (C to S)	{token}	Must arrive within 5 s of open; token verified; on success server replies auth:ok and broadcasts presence ONLINE	AUTH_FAILED, close code 4401

Event (direction)	Payload	Validation and behaviour	Error codes
message:send (C to S)	{conversationId, content, clientMsgId}	content 1 to 2,000 chars, not blank; sender must be a member; sanitised; rate limit 20 per 10 s; stored before SENT ack	VALIDATION_ERROR, MESSAGE_TOO_LONG, FORBIDDEN_NOT_MEMBER, RATE_LIMITED
message:new (S to C)	{messageId, conversationId, senderId, senderName, content, sentAt}	Pushed to all online members except the sender; offline members get it on resync	none
message:status (both)	{messageId, status}	Server to sender: SENT, DELIVERED, READ. Recipient to server: DELIVERED or READ; only the recipient may raise status; status never goes backwards	FORBIDDEN_NOT_MEMBER, VALIDATION_ERROR
typing (both)	{conversationId, userId, isTyping}	Relayed to other members; client clears indicator after 3 s of inactivity	FORBIDDEN_NOT_MEMBER
presence (S to C)	{userId, status, lastSeen}	Sent to contacts within 2 s of connect or disconnect	none
group:update (S to C)	{conversationId, change, userId}	change = ADDED, REMOVED, RENAMED or LEFT	none
error (S to C)	{code, message}	Reports rejected input or authorisation failure; the connection stays open unless the error is AUTH_FAILED	see 4.4

4.4 Error Handling, Logging & Monitoring

Standard error format. REST errors use the envelope {"error": {"code", "message", "requestId"}} and WebSocket errors use the event {"type": "error", "payload": {"code", "message"}}. Messages are plain language and never contain stack traces, SQL/NoSQL details or secrets.

Code	HTTP	Meaning	User-visible message
VALIDATION_ERROR	400	Field missing, wrong format, weak password, blank message	The specific rule that failed, next to the field
MESSAGE_TOO_LONG	400	Content over 2,000 characters	Message is too long (maximum 2,000 characters)
AUTH_INVALID_CREDENTIALS	401	Wrong username/email or password	Invalid credentials
AUTH_TOKEN_INVALID	401	Missing, malformed or expired token	Please log in again
FORBIDDEN_NOT_MEMBER	403	User is not a member of the conversation	You do not have access to this conversation
FORBIDDEN_NOT_ADMIN	403	Group management by a non-admin	Only the group admin can do this
NOT_FOUND	404	User or conversation does not exist	Not found
DUPLICATE_USER	409	Username or email already registered	Username or email is already in use
ACCOUNT_LOCKED	423	5 failed logins, locked for 15 minutes	Account locked, try again later
RATE_LIMITED	429	More than 20 messages in 10 seconds	You are sending messages too fast
INTERNAL_ERROR	500	Unexpected server fault	Something went wrong, please try again
SERVICE_UNAVAILABLE	503	Database or WebSocket server unavailable	Service unavailable, retrying...

Handling strategy.

- A central Express error-handling middleware and a try/catch wrapper around every WebSocket handler convert exceptions into the standard error format; unhandled rejections are logged and the process stays up.

- If MongoDB is unreachable the server returns SERVICE_UNAVAILABLE, does not acknowledge the message, and keeps no partial writes; the client shows a toast and keeps the unsent text so that the user can retry (SAF-3).
- Client network loss is not an error: the client shows "Reconnecting...", retries with exponential back-off and re-synchronises (REQ-22).
- Duplicate sends caused by retries are ignored using clientMsgId and messageId.

Logging. Structured JSON logs with level (debug, info, warn, error), timestamp, requestId and userId. Logged events: login success/failure, account lock, WebSocket open/close, rejected token, rate-limit hits, errors. **Never logged:** passwords, password hashes, JWTs, message content (SEC-2).

Monitoring. GET /health returns status and uptime and checks the database connection. Counters kept in memory and printed periodically or exposed to the administrator: active WebSocket connections, messages per minute, p95 delivery latency (PERF-1), failed-login count, error rate.

4.5 UX Design

- Single-page application with the screens defined in SRS 3.1: Register/Login, Chat Home (conversation list with unread badges and online users on the left, message pane in the centre, input at the bottom), Create/Manage Group, Profile/Settings and toast notifications.
- Responsive layout from 360 px; on narrow screens the sidebar and message pane are shown one at a time with a back button.
- Own messages are right-aligned and coloured differently from received messages; status ticks (one tick sent, double tick delivered, blue double tick read).
- Keyboard shortcuts: Enter sends, Shift+Enter inserts a line, Esc closes a dialog.
- Visible connection-status indicator and "<name> is typing..." line; browser tab title shows the unread count.
- Error messages in plain language next to the field or as a toast (red = error, green = success); sufficient colour contrast and labelled inputs.
- Target: a new user can register and send a first message within 2 minutes (SRS 6.4 Usability). Wireframes will be added to the repository during sprint 1.

4.6 Open Issues & Next Steps

- Confirm with the guide whether READ status is required for group messages; this design keeps SENT only for groups, as in REQ-25.
- Decide the logging library (pino or winston) and the in-memory versus Redis-backed rate limiter before sprint 2.
- Prepare wireframes and a clickable prototype of Chat Home.
- Create the repository skeleton (module folders, ESLint, .env.example, npm start) and start sprint 1 with Auth and Messaging.
- Future enhancements: file sharing, message editing, end-to-end encryption, horizontal scaling with a message broker, push notifications.

5. Appendices

5.1 Glossary: see Section 1.4 of this document and Appendix A of the SRS.

5.2 References:

- IEEE Std 1016 / ISO/IEC/IEEE 42010, Architecture description and software design practice.
- Software Requirements Specification, Chat Application, Team 5, v1.0, 04-Oct-2026.
- RFC 6455, The WebSocket Protocol; RFC 7519, JSON Web Token.
- OWASP Top 10 Web Application Security Risks; OWASP WebSocket Security Cheat Sheet.

- Node.js, Express, ws, Mongoose and bcrypt documentation.

5.3 Tools: Graphviz (component diagram), Mermaid (sequence diagrams), draw.io / PlantUML (optional), Postman and Swagger/OpenAPI (API contract), Git, ESLint.